SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: Deep Learning **COURSE CODE: MLA0405**
FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

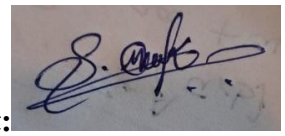
Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I Shaik Mubarak(192425194)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

QUESTIONS: **Total Marks:25**

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

a) Identify the conceptual error in the likelihood formulation.
b) Explain how this mistake affects parameter estimation.
c) Suggest the correct mathematical formulation and reasoning.
d) How would using log-likelihood fix computational issues?

Answers:

Problem Identification

A binary classifier is implemented using Maximum Likelihood Estimation (MLE). However, the likelihood function is incorrectly written as the sum of probabilities instead of the product of probabilities, resulting in poor parameter estimation even with a large dataset.

a) Conceptual Error

For independent observations, the likelihood function must represent the joint probability of all samples.

The student incorrectly used

$$L(\theta) = P(x_1) + P(x_2) + \cdots + P(x_n)$$

This is mathematically incorrect because probabilities of independent events are multiplied, not added.

The correct likelihood function is

$$L(\theta) = \prod_{i=1}^{n} P(x_i \mid \theta)$$

b) Effect on Parameter Estimation

Since the likelihood function is incorrectly formulated,

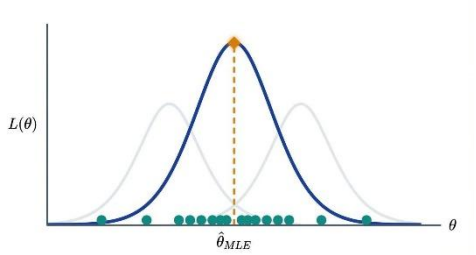
- The optimization objective becomes incorrect.
- The estimated parameter no longer maximizes the true probability of observing the data.
- Gradient calculations become incorrect.
- The model converges to inaccurate parameter values.
- Prediction accuracy decreases even when more training samples are provided.

c) Correct Mathematical Formulation

For a binary classifier,

$$L(\theta) = \prod_{i=1}^{n} P(y_i \mid x_i, \theta)$$

Taking logarithm,



$$\log L(\theta) = \sum_{i=1}^{n} \log P(y_i \mid x_i, \theta)$$

The optimization algorithm maximizes this log-likelihood to obtain the optimal parameter values.

d) Why Log-Likelihood is Preferred

Direct multiplication of many probabilities produces extremely small values.
Example:

$$0.8 \times 0.7 \times 0.9 \times 0.6 = 0.3024$$

For thousands of samples,

$$0.9^{1000} \approx 0$$

leading to numerical underflow.
Using logarithms,

$$\log(ab) = \log a + \log b$$

Thus,

$$\log L = \sum_{i=1}^{n} \log P(y_i \mid x_i)$$

This avoids underflow and simplifies differentiation during optimization.

Conclusion

Using the product of probabilities correctly represents the likelihood of the complete dataset. Applying log-likelihood improves numerical stability, simplifies optimization, and produces accurate parameter estimates.

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

a) List possible implementation errors in the learning algorithm.
b) Analyze how incorrect learning rate or weight updates affect convergence.
c) Identify dataset-related issues that may cause this problem.
d) Propose modifications to ensure convergence.

Answer:

Problem Identification

A Perceptron does not converge even though the dataset is linearly separable.

a) Possible Implementation Errors

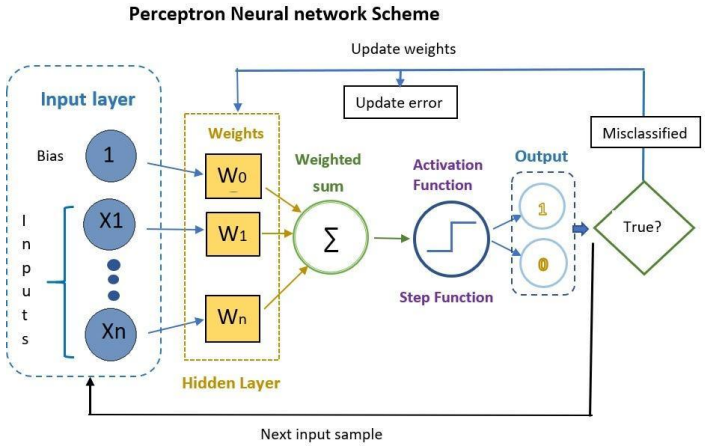
Possible errors include:

- Incorrect weight initialization.
- Incorrect bias update.
- Wrong sign in the weight update equation.

- Incorrect activation function implementation.
- Labels encoded incorrectly (0/1 instead of −1/+1 when required).
- Missing bias term.

Correct weight update:

$$w_{new} = w_{old} + \eta(y - \hat{y})x$$



b) Effect of Incorrect Learning Rate

If

$$\eta = 2$$

updates become excessively large.
Weights oscillate around the decision boundary.
If

$$\eta = 0.0001$$

updates become extremely small.
Training becomes very slow.
Therefore,
An appropriate learning rate balances convergence speed and stability.

c) Dataset Issues

Even though the dataset is assumed linearly separable, convergence may fail if:

- Labels are incorrect.
- Features are not normalized.
- Duplicate or noisy samples exist.
- Missing values are present.

These issues confuse the learning algorithm.

d) Modifications

To ensure convergence:

- Normalize input features.
- Verify label encoding.
- Include bias.
- Use an appropriate learning rate.
- Shuffle training samples each epoch.
- Verify the weight update implementation.

Conclusion

Correct implementation, proper preprocessing, and suitable hyperparameters ensure successful convergence of the Perceptron Learning Algorithm.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

a) Identify possible mathematical or coding errors in gradient computation.
b) Explain the role of activation functions in gradient flow.
c) Analyze how vanishing or exploding gradients affect training.
d) Suggest techniques to fix the issue (e.g., normalization, initialization).

Answers:

Problem Identification

During training, the MLP loss continuously increases instead of decreasing.

a) Possible Gradient Errors

Possible causes include:

- Incorrect derivative calculation.
- Wrong chain rule implementation.
- Gradient sign reversed.
- Incorrect learning rate.
- Errors in matrix multiplication.
- Incorrect loss function implementation.

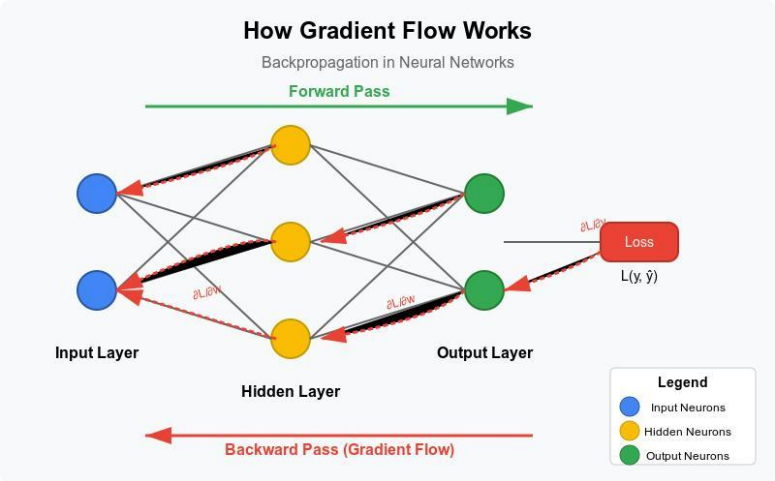
Gradient update should be

$$W = W - \eta \frac{\partial J}{\partial W}$$

If the sign becomes

$$W = W + \eta \frac{\partial J}{\partial W}$$

the model performs gradient ascent instead of descent, increasing the loss.



b) Role of Activation Functions

Activation functions introduce nonlinearity.
During backpropagation,

$$\delta = f'(z)$$

If

$$f'(z) \approx 0$$

gradient flow becomes weak.
Sigmoid activation often produces very small gradients for large positive or negative inputs.

c) Vanishing and Exploding Gradients
Vanishing Gradient:
Repeated multiplication of values less than one

$$0.5 \times 0.5 \times 0.5 = 0.125$$

Gradient approaches zero.
Weights stop updating.
Exploding Gradient:
Repeated multiplication of values greater than one

$$2 \times 2 \times 2 = 8$$

Gradient becomes extremely large.
Training becomes unstable.

d) Solutions
Recommended techniques:

- Xavier initialization.
- He initialization.
- Batch Normalization.
- Gradient Clipping.
- ReLU activation.
- Proper learning rate.

These techniques stabilize gradient flow and improve convergence.

Conclusion

Correct gradient computation, suitable activation functions, proper initialization, and normalization ensure stable backpropagation and decreasing training loss.

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.
Tasks:
a) Explain why batch gradient descent is slow in large datasets.
b) Analyze the cause of fluctuations in SGD.
c) Compare the error behavior of Mini-Batch Gradient Descent.
d) Recommend the best approach and justify with error trade-offs.

Answers:

Problem Identification

Batch Gradient Descent converges slowly, whereas SGD exhibits unstable loss fluctuations.

a) Why Batch Gradient Descent is Slow
Gradient computation uses the entire dataset.
Gradient equation:

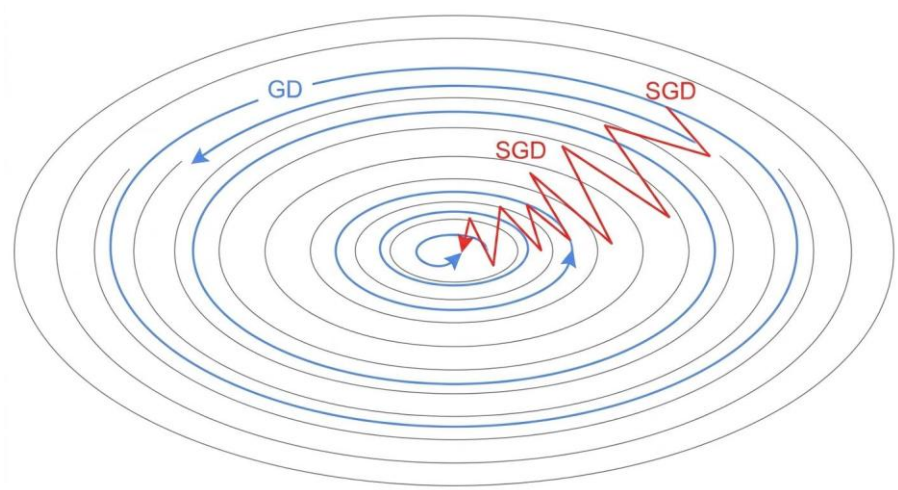
$$\nabla J = \frac{1}{n}\sum_{i=1}^{n} \nabla J_i$$

For large datasets,
Every iteration processes all training samples.
Training time increases significantly.

b) Cause of SGD Fluctuations
SGD updates after every sample.
Update equation:

$$w = w - \eta \nabla J_i$$

Each sample provides a different gradient.
Therefore,
Loss fluctuates because updates depend on individual samples.



c) Mini-Batch Gradient Descent
Mini-Batch uses a small batch.
Example:
Batch size = 32.
Gradient

$$\nabla J = \frac{1}{32} \sum_{i=1}^{32} \nabla J_i$$

Advantages:

- Faster than Batch GD.
- More stable than SGD.
- Efficient GPU utilization.
- Lower variance.
-

d) Recommended Approach

Mini-Batch Gradient Descent provides the best compromise.

Method	**Speed**	**Stability**
Batch GD	Slow	Very Stable
SGD	Fast	Unstable
Mini-Batch	Moderate	Stable

Conclusion

Mini-Batch Gradient Descent balances computational efficiency, convergence speed, and optimization stability, making it the preferred approach for most deep learning applications.

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

a) Explain how high dimensionality increases model error.
b) Identify signs of overfitting in this scenario.
c) Analyze why distance-based learning fails in high dimensions.
d) Suggest dimensionality reduction or regularization techniques to improve performance.

Answers:

Problem Identification

The model achieves high training accuracy but poor testing accuracy because of high-dimensional data.

a) How High Dimensionality Increases Error

Suppose
Features

$$100 \rightarrow 1000 \rightarrow 10000$$

The feature space expands exponentially.
Training samples become sparse.
The model memorizes noise instead of learning meaningful patterns.
Prediction error on new data increases.

b) Signs of Overfitting

Typical observations:
Training Accuracy

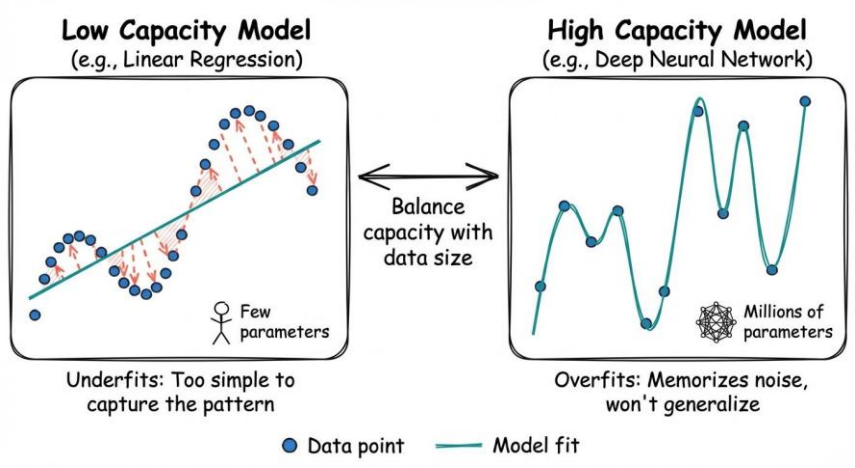
$$99\%$$

Testing Accuracy

$$72\%$$

Large accuracy gap indicates overfitting.
Other signs include:

- Very low training loss.
- High validation loss.
- Poor generalization.



-
-

c) Why Distance-Based Learning Fails

Distance between samples becomes nearly identical.

Example:

Nearest distance

$$10.2$$

Farthest distance

$$10.8$$

Difference

$$0.6$$

Algorithms such as **KNN** cannot distinguish neighboring samples effectively because distance loses discriminative power in high-dimensional spaces.

d) Improvement Techniques

Principal Component Analysis (PCA)

Reduces

$$10000 \rightarrow 300$$

important features while preserving most variance.

L2 Regularization

$$J(W) = Loss + \lambda \,||\, W \,||^2$$

Penalizes large weights and reduces model complexity.

Other techniques include:

- Feature Selection
- Dropout
- Early Stopping
-

Conclusion

High dimensionality increases model complexity, weakens distance-based learning, and causes overfitting. Applying dimensionality reduction and regularization improves generalization, reduces prediction error, and enhances model performance.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand